SENG401 – GRADUATION PROJECT

Software Design Document

By Sinem Coşkun, Al-baraa Abdullah Abdullah Abdullah Al-yadumi

TABLE OF CONTENTS

1. Introduction 3
1.1 Purpose of the SDD 3
1.2 Scope 3
1.3 Definitions, Acronyms, and Abbreviations 3
1.4 References 4
1.5 Document Overview 4
2. System Overview 5
2.1 High-Level System Description 5
2.2 Major System Components 5
2.3 Design Constraints and Assumptions 7
2.4 High-Level Architecture Diagram 8
3. Architectural Design 10
3.1 Architectural Style and Layers 10
3.2 Subsystem Decomposition 10
3.2.1 Attacker Layer (External Actor) 10
3.2.2 Honeypot Layer 11
3.2.3 Data Transfer and Storage Layer 11
3.2.4 Statistical Analysis Layer 12
3.2.5 Machine Learning Analysis Layer 12
3.2.6 Visualization Layer 13
3.3 Component Interaction Overview 13
3.4 Architectural Diagrams: 14
4. Data Design 16
4.1 Data Flow Overview 16
4.2 Logical Data Model — SQLite 18
4.2.1 Raw Log Tables 18
4.2.2 Preprocessed Data Tables 19
4.2.3 Feature Tables 20
4.3 Logical Data Model — InfluxDB (Time-Series Storage) 20
4.3.1 Tunneling Results 20
4.3.2 Classification Results 20
4.3.3 Performance Metrics 21

4.4 Data Flow Diagram (DFD) 21
4.5 Data Dictionary (High-Level) 22
5. Component Design 23
5.1 Honeypot Layer Components 24
5.1.1 Cowrie Component (SSH Honeypot) 24
5.1.2 Honeytrap Component (HTTP/FTP Emulation) 24
5.1.3 Suricata Component (Network IDS) 24
5.2 Data Transfer and Storage Components 25
5.2.1 Log Aggregation Component 25
5.2.2 MQTT Publisher Component (Edge Device) 25
5.2.3 MQTT Broker Component 26
5.2.4 SQLite Storage Component 26
5.3 Statistical Analysis Components (Tunneling Detection) 26
5.3.1 Entropy Calculation Component 26
5.3.2 Sliding Window Component 27
5.3.3 Threshold Decision Component 27
5.4 Machine Learning Analysis Components 27
5.4.1 Preprocessing Component 27
5.4.2 Feature Extraction Component 28
5.4.3 Classification Component 28
5.5 Visualization Components 29
5.5.1 InfluxDB Time-Series Storage Component 29
5.5.2 Dashboard Visualization Component (Grafana / Superset) 29
6. Algorithm Design 30
6.1 Entropy Calculation Algorithm 30
6.2 Sliding Window Algorithm 31
6.3 Tunneling Decision Algorithm 32
6.4 Machine Learning Classification Workflow 33
6.4.1 Feature Engineering Algorithm 34
6.4.2 Classification Algorithm 34
6.5 Algorithm Dependencies 35
7. User Interface Design 35
7.1 Dashboard Overview 35

7.2 Dashboard Layout and Panels 36
7.3 Interaction Scenarios 37
7.4 UI Wireframe / Diagram 37
8. Non-Functional Design 38
8.1 Performance Design 38
8.2 Reliability Design 38
8.3 Security Design 39
8.4 Usability Design 39
8.5 Maintainability Design 40
8.6 Extensibility Design 40
8.7 Portability Design 40
9. Deployment & Runtime View 41
9.1 Physical Deployment Diagram 41
9.2 Runtime Scenarios (Sequence Diagrams) 42
9.2.1 Runtime Scenario 1 — Attack Event to Dashboard (Full Pipeline) 42
9.2.2 Runtime Scenario 2 — DNS/HTTP Tunneling Detection Flow 43
9.2.3 Runtime Scenario 3 — Machine Learning Attack Classification Flow 43
10. SRS Traceability 44
10.1 Functional Requirements Traceability 44
10.2 Non-Functional Requirements Traceability 46

1. Introduction

1.1 Purpose of the SDD

The purpose of this Software Design Document (SDD) is to provide a detailed technical description of the system design for the "Detecting Protocol Tunneling Activities and Cyber Attacks with Machine Learning-Based IoT Honeypot." project.
This document translates the functional and non-functional requirements defined in the SRS into a comprehensive design plan that guides implementation, integration, testing, and deployment. The SDD ensures consistent and sustainable configuration of all components, including the honeypot layer, data acquisition, preprocessing, feature extraction, statistical detection, machine learning, storage, and visualization.

1.2 Scope

This SDD covers the entire design of the IoT honeypot-based detection system described in the SRS. It includes architectural design, subsystem descriptions, data models, algorithms, interaction flows, sequencing diagrams, deployment views, and mappings to SRS requirements.

Here's what the design covers:

- It runs an IoT honeypot right on the edge device.
- Logs get collected and sent out using MQTT.
- All those logs end up in a central SQLite database.
- Before any analysis, it preprocesses the data and pulls out features to spot tunneling and attacks.
- For DNS and HTTP traffic, the system uses entropy to catch tunneling.
- Provides Machine Learning classification of cyber attacks.
- Time-series storage and dashboard visualization using InfluxDB & Grafana/Superset.

This document only reflects capabilities defined in the SRS and does not introduce new functionalities.

1.3 Definitions, Acronyms, and Abbreviations

IoT: Internet of Things
DMZ: Demilitarized Zone (isolated network for honeypots)
Honeypot: A fake system built to lure hackers and record what they do
MQTT: Message Queuing Telemetry Transport
SQLite: Lightweight relational database for log/event storage
InfluxDB: Time-series database used for storing detection outputs
DNS: Domain Name System
HTTP: Hypertext Transfer Protocol
ML: Machine Learning
Entropy: A way to measure how random data is, which helps spot tunneling

Feature Extraction: Turning raw log data into useful numbers for analysis
Grafana / Superset: Visualization tools
Brute Force Attack: Trying lots of passwords over and over to break in
Port Scan: Systematic scanning of open ports
Malware Download: Trying to download or install malicious files
Bot vs Human Behavior: Figuring out if actions come from a person or a bot

1.4 References

[R-SRS] Software Requirements Specification (SRS) for "Detecting Protocol Tunneling Activities and Cyber Attacks with Machine Learning-Based IoT Honeypot", 2025.

[1] F. Al-Turjman et al., "Federated Learning for IoT Security and Privacy," Future Internet, 2025.

[2] M. Kaur and S. Singh, "IoT Network Security Threats and Countermeasures," Computers, 2025.

[3] R. N. Rao, "AI-Driven Detection of IoT Attacks using Honeypot Data," PMC, 2024.

[4] A. Ahmed et al., "A Honeypot-Driven Approach to IoT Attack Detection," IET Smart Cities, 2024.

[5] R. D. Venkat Goundla, "Analysis of IoT Honeypots for Network Security," Applied Sciences, 2025.

[6] V. G. Goundla, "Optimizing IoT Honeypots with Machine Learning," NCI Repository, 2024.

[7] D. Ivanov, "Securing Military Embedded Systems: Three Challenges," Military Embedded Systems, 2024.

[8] Quest Defense, "Evolution of Embedded Systems in Aerospace & Defense," Defense Insights, 2024.

[9] X. Huang and Y. Huo, "A Review of Tunnel Detection on Multilayer Protocols," Applied Sciences, 2023.

[10] Alve, "Smart IoT Security: Lightweight Machine Learning Techniques," arXiv, 2025.

[11] A. Bhandari et al., "Distributed Deep Learning for IoT Cyber-Attack Detection," Electronics, 2023.

[12] T. Salat et al., "DNS Tunnelling, Exfiltration and Detection in Cloud Environments," Sensors, 2023.

[13] F. Üstebay, "Enhancing Zero-Day Attack Detection in IoT Networks via Isolation," Electrica, 025.1.5 Document Overview

1.5 Document Overview

The remainder of this SDD is structured as follows:

- **Section 2:** High-level system description and architecture overview.
- **Section 3:** Architectural design, subsystem decomposition, component interactions, and diagrams.
- **Section 4:** Data design, logical models, and data flow diagrams.

- **Section 5:** Component-level design for each subsystem.
- **Section 6:** Algorithms for entropy, sliding-window detection, tunneling logic, and ML workflows.
- **Section 7:** Visualization and dashboard design.
- **Section 8:** Non-functional design considerations.
- **Section 9:** Deployment model and runtime sequence diagrams.
- **Section 10:** Traceability between SRS requirements and SDD design elements.

2. System Overview

2.1 High-Level System Description

The system is an IoT-based cyber-attack monitoring and analysis platform designed to detect protocol tunneling activities and classify various cyber attacks using statistical analysis and machine-learning techniques.
A Raspberry Pi device acts as the edge honeypot, exposed to the Internet through a DMZ/guest network, allowing it to receive real-world malicious traffic.
Three open-source security tools—Cowrie, Honeytrap, and Suricata—simulate vulnerable services and capture attacker interactions, commands, sessions, and packet-level activity.

All collected logs are transferred to a central analysis machine through the lightweight MQTT protocol and stored in a SQLite database.
The data is then preprocessed, and traffic features such as entropy, URI length, query frequency, session duration, connection intensity, and IP diversity are extracted.

Two complementary analysis pipelines operate on this data:

1. Statistical Tunneling Detection
   Using Shannon entropy and sliding-window analysis to detect DNS and HTTP tunneling.
2. Machine-Learning-Based Attack Classification
   Identifying brute-force attempts, port scans, malware downloads, and bot-vs-human behavior using supervised and anomaly-detection models.

Final results are stored in InfluxDB, a time-series database, and visualized through Grafana or Apache Superset dashboards.

2.2 Major System Components

1. Attacker (External Actor)

- Source of real-world cyber attacks
- Initiates SSH, HTTP/FTP, DNS and other network traffic

2. Honeypot Layer

Components:

- Cowrie
- Honeytrap
- Suricata

Responsible for:

- Capturing commands, sessions, requests, network alerts
- Generating time-stamped raw logs

3. Data Transfer and Storage Layer

Components:

- Log Aggregator (Raspberry Pi)
- MQTT Publisher (Raspberry Pi)
- MQTT Broker (Central Machine)
- SQLite Database

Responsible for:

- Standardizing logs
- Publishing logs to MQTT
- Receiving and storing logs in SQLite

4. Statistical Analysis Layer

Components:

- Entropy Calculation Module
- Sliding-Window Module
- Threshold Decision Engine

Responsible for:

- Computing entropy values
- Analyzing DNS/HTTP sequences
- Detecting tunneling vs normal traffic

5. Machine Learning Analysis Layer

Components:

- Preprocessing Module
- Feature Extraction Module
- ML Classification Module

Responsible for:

- Computing features (URI length, entropy, session duration…)
- Training classifiers
- Labeling attacks and measuring performance metrics

6. Visualization Layer

Components:

- InfluxDB Time-Series Storage
- Grafana / Apache Superset Dashboards

Responsible for:

- Storing analysis results
- Presenting visual dashboards for research and analysis

2.3 Design Constraints and Assumptions

The system is designed under the following constraints derived from the SRS:

Hardware Constraints

- Raspberry Pi 4 Model B (limited CPU, RAM, and storage)
- Must run honeypots continuously while exposed to the Internet

Network Constraints

- Raspberry Pi must be placed in DMZ / Guest Network
- No connection to production or university networks

Software and Tooling Constraints

- Only open-source tools may be used:
    - Cowrie, Honeytrap, Suricata
    - MQTT
    - SQLite
    - Python (Pandas, Scikit-learn)
    - InfluxDB
    - Grafana / Superset

Analytical Constraints

- Tunneling detection must reach:
    - **≥ 80% accuracy**
- ML classifiers must reach:
    - **≥ 80% F1-score**

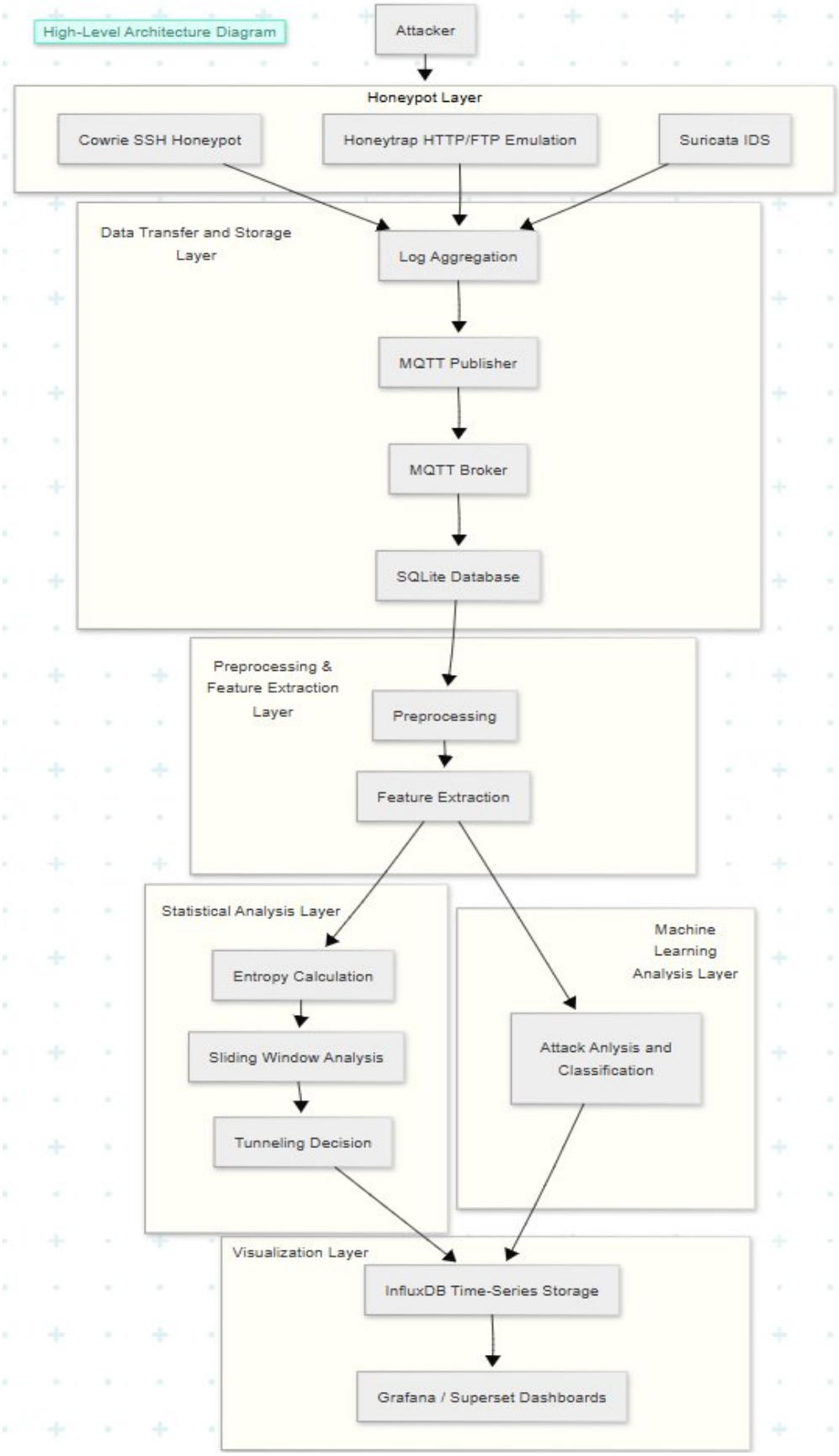
Operational Assumptions

- Raspberry Pi will receive sufficient real attack traffic
- MQTT communication remains stable
- SQLite can handle log volume
- Python scripts run on central machine
- Dashboards correctly connect to InfluxDB

2.4 High-Level Architecture Diagram

This subsection will contain the main system diagram describing:

- Attacker → Honeypot → Log Collection → MQTT → SQLite → Analysis → InfluxDB → Dashboard


High-Level Architecture Diagram
Attacker
Honeypot Layer
Cowrie SSH Honeypot
Honeytrap HTTP/FTP Emulation
Suricata IDS
Data Transfer and Storage Layer
Log Aggregation
MQTT Publisher
MQTT Broker
SQLite Database
Preprocessing & Feature Extraction Layer
Preprocessing
Feature Extraction
Statistical Analysis Layer
Entropy Calculation
Sliding Window Analysis
Tunneling Decision
Machine Learning Analysis Layer
Attack Anlysis and Classification
Visualization Layer
InfluxDB Time-Series Storage
Grafana / Superset Dashboards

3. Architectural Design

In this section, the architectural structure of the system is explained using a layered model.
The design reflects how data flows from external attackers to honeypot servers, storage, analysis modules, and visualization tools.
All architectural details are strictly derived from the SRS.

3.1 Architectural Style and Layers

The system follows a **layered architecture** where each layer has a separate responsibility in the overall data processing and analysis process.

The architectural layers are:

1. **Attacker Layer**
2. **Honeypot Layer**
3. **Data Transfer and Storage Layer**
4. **Statistical Analysis Layer**
5. **Machine Learning Analysis Layer**
6. **Visualization Layer**

Each layer processes specific inputs and produces outputs that flow to the next layer.

The design ensures modularity, ease of maintenance, clear separation of responsibilities, and ease of future expansion.

3.2 Subsystem Decomposition

This section describes each architectural subsystem in detail.

3.2.1 Attacker Layer (External Actor)

Role:
It represents real-world attackers who generate malicious traffic targeting the Raspberry Pi device.

Responsibilities:

- Initiate SSH brute-force attempts
- Send HTTP/FTP requests
- Perform port scans
- Generate DNS and HTTP traffic used for tunneling detection
- Attempt malware-related actions

Outputs:

- Malicious traffic reaching the Honeypot Layer

3.2.2 Honeypot Layer

This layer simulates vulnerable services to capture attacker interactions.

Components:

- **Cowrie** — SSH/Telnet honeypot
- **Honeytrap** — HTTP/FTP service emulation
- **Suricata** — IDS for packet inspection

Responsibilities:

- Interact with attackers
- Log commands, sessions, HTTP requests, and IDS alerts
- Timestamp all events
- Generate protocol-specific log files

Outputs:

- Raw log entries (SSH, HTTP/FTP, network alerts)

Consumes:

- Incoming attacker traffic

3.2.3 Data Transfer and Storage Layer

This layer handles log aggregation, reliable transfer, and structured storage.

Components:

- Log Aggregator (Raspberry Pi)
- MQTT Publisher (Raspberry Pi)
- MQTT Broker (Central Machine)
- SQLite Database

Responsibilities:

- Collect logs from honeypots
- Standardize log format

- Forward logs using MQTT
- Receive logs at the central server
- Parse and validate log entries
- Store logs and extracted features in SQLite

Outputs:

- Structured log datasets

Consumes:

- Honeypot-generated logs

3.2.4 Statistical Analysis Layer

This subsystem performs detection of DNS/HTTP tunneling.

Components:

- Entropy Calculation Module
- Sliding Window Module
- Threshold Decision Engine

Responsibilities:

- Compute Shannon entropy per traffic segment
- Analyze patterns using sliding windows
- Apply threshold-based classification
- Label traffic as "Tunneling" or "Normal"

Outputs:

- Tunneling detection results

Consumes:

- Preprocessed DNS/HTTP logs
- Extracted entropy-based features

3.2.5 Machine Learning Analysis Layer

This subsystem performs attack classification using supervised or anomaly-detection techniques.

Components:

- Preprocessing Module
- Feature Extraction Module
- ML Classification Module

Responsibilities:

- Clean and normalize data
- Extract features such as entropy, URI length, session duration, etc.
- Train classification models
- Detect brute force, port scanning, malware download attempts, and bot/human behavior
- Compute performance metrics (accuracy, F1-score, etc.)

Outputs:

- Attack labels
- Classification metrics

Consumes:

- SQLite-stored logs and features

3.2.6 Visualization Layer

This subsystem presents analysis outputs to the user.

Components:

- InfluxDB Time-Series Storage
- Grafana or Apache Superset Dashboards

Responsibilities:

- Store tunneling and ML results as time-series data
- Generate visual dashboards showing:
    - tunneling detections
    - classified attacks
    - geographic origins

    - bot vs human behavior
    - accuracy, F1-score, false positive rate

Outputs:

- Interactive dashboards
- Time-based visualizations

Consumes:

- Analysis results (tunneling + ML classification)

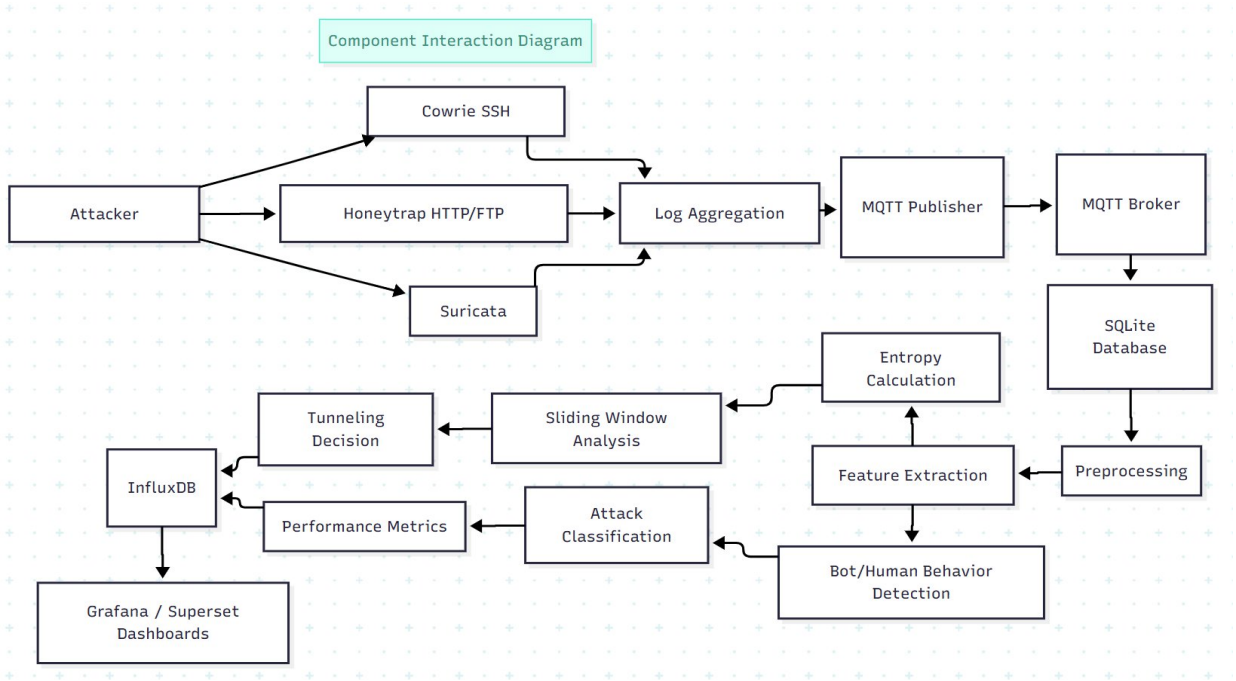
3.3 Component Interaction Overview

This section summarizes how the architectural subsystems interact:

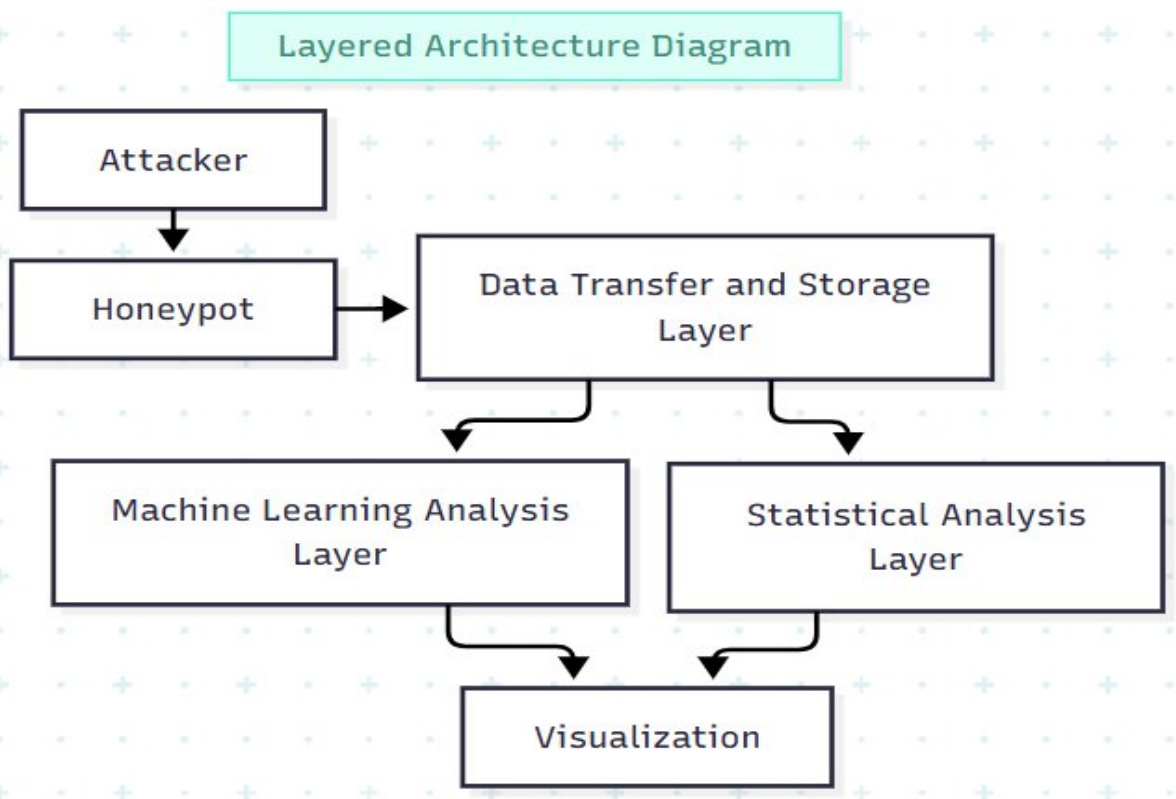
1. **Attackers** generate malicious traffic toward the Raspberry Pi.
2. **Honeypot Layer** captures attacker interactions and generates logs.
3. **Log Aggregator** standardizes logs.
4. **MQTT Publisher** transmits logs to the central server.
5. **MQTT Broker** receives logs and forwards them.
6. **SQLite Storage** organizes logs for analysis.
7. **Statistical Analysis Layer** uses entropy and sliding-window techniques to detect tunneling.
8. **ML Analysis Layer** extracts features and classifies attack types.
9. **InfluxDB** stores final results as time-series data.
10. **Grafana/Superset** dashboards show visual summaries.

3.4 Architectural Diagrams:

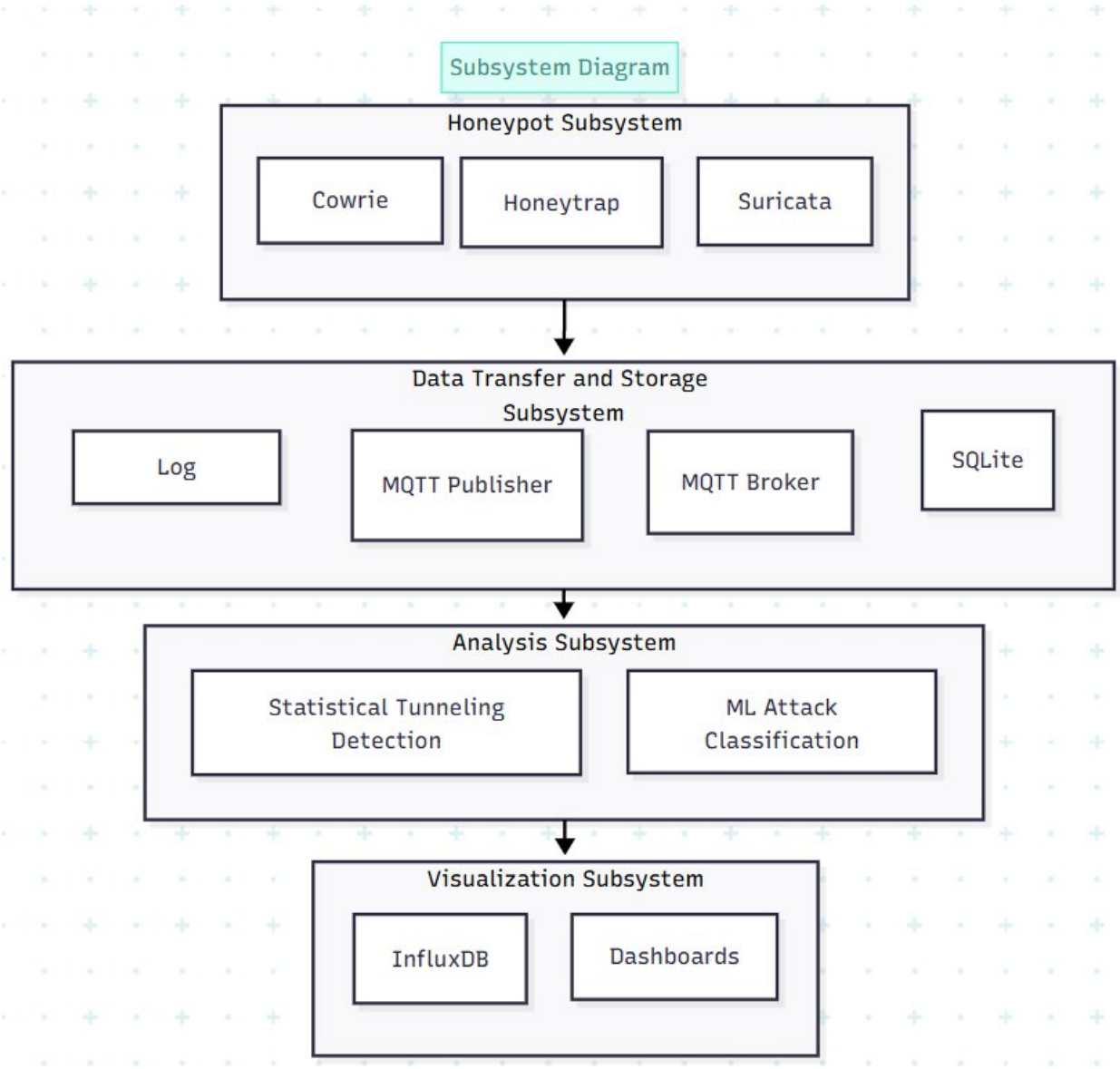
Component Interaction Diagram:



Layered Architecture Diagram:



Subsystem Diagram:



4. Data Design

This section describes how data is collected, transmitted, stored, transformed, and used across the system. All structures directly follow the SRS without adding any new elements.

4.1 Data Flow Overview

The overall data lifecycle in the system follows these stages:

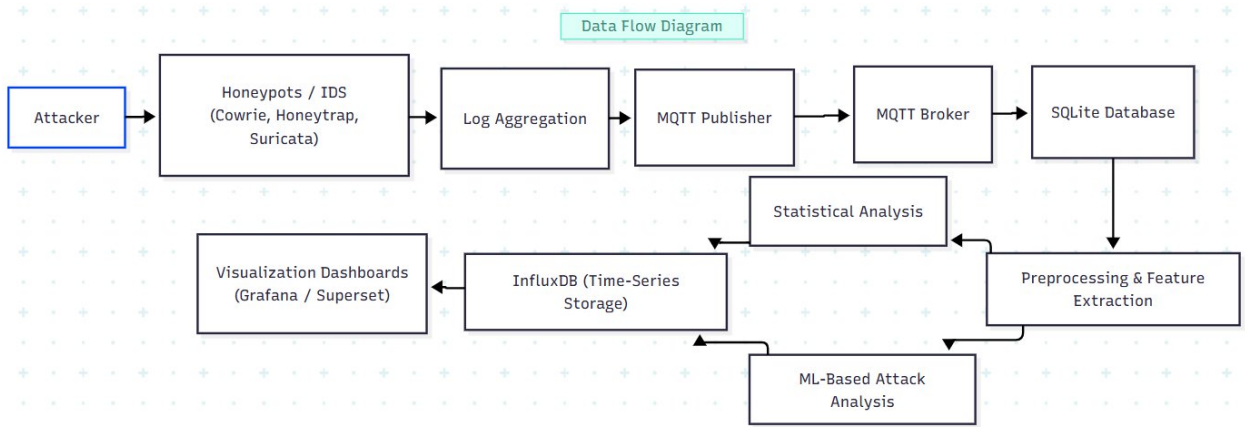
1. **Attackers generate malicious traffic**
   (SSH, HTTP/FTP, DNS, port scans, malware-related traffic)

2. **Honeypot Layer records raw logs**

- Cowrie SSH commands, session details
- Honeytrap HTTP/FTP interactions
- Suricata IDS alerts and packet metadata

3. **Logs are aggregated and prepared on the Raspberry Pi**
    - Standardization and Metadata addition (timestamp, source, event type)
4. **MQTT transfers logs to the central server**
5. **SQLite stores structured log records**
    - Raw logs
    - Preprocessed logs and Extracted features
6. **Preprocessing extracts relevant fields**
    - normalization and field extraction
    - conversion to analysis-ready form
7. **Feature extraction generates numerical attributes**
    - entropy
    - session duration and query frequency
    - URI length
    - IP diversity and connection intensity
8. **Statistical and ML analysis modules process the features**
9. **InfluxDB stores analysis outputs as time-series entries**
10. **Grafana/Superset generate dashboards**

The data flow diagram showing this entire structure is below:



4.2 Logical Data Model — SQLite

The SQLite database stores three main categories of data:

1. Raw Log Tables

Derived directly from Honeypot Layer logs.

2. Preprocessed Data Tables

Cleaned and normalized logs.

3. Feature Tables

Numeric features used for tunneling detection and ML classification.

Below is the logical structure (not physical SQL schema; no implementation details, as required by SRS).

4.2.1 Raw Log Tables

Cowrie Log

Field	Description
timestamp	Exact time of SSH interaction
source_ip	Attacker IP address
command	Command entered by attacker
session_id	Unique session identifier
event_type	Login attempt, command input, etc.

Honeytrap Log

Field	Description
timestamp	Time of HTTP/FTP request
method	HTTP method (GET/POST/etc.)
uri	Request URI
source_ip	Attacker IP
user_agent	User agent / client string
event_type	Type of service interaction

Suricata Log

Field	Description
timestamp	Time of IDS alert
alert_type	Signature or rule triggered
source_ip	Origin of packet
destination_ip	Target IP (honeypot)
protocol	DNS/HTTP/TCP/UDP
metadata	Additional Suricata event data

4.2.2 Preprocessed Data Tables

Preprocessed_Log

Field	Description
timestamp	Cleaned timestamp
log_source	Cowrie / Honeytrap / Suricata
session_id	Normalized session identifier
request_data	Parsed request/command data
dns_query	Extracted DNS fields (if applicable)
http_uri	Extracted URI fields

Purpose:
Used directly for tunneling detection and ML feature extraction.

4.2.3 Feature Tables

Extracted_Features

Feature	Description
session_duration	Duration of attacker's session
query_frequency	Number of requests over time
character_entropy	Shannon entropy score

Feature	Description
uri_length	Length of HTTP URI
ip_diversity	Count of distinct IPs per time window
connection_intensity	Rate of requests/packets
label (if available)	Normal / Attack Type / Bot-Human

Used by:

- Tunneling detection
- ML classification models

4.3 Logical Data Model — InfluxDB (Time-Series Storage)

InfluxDB stores final analysis results.
Each entry is time-stamped and indexed for visualization.

4.3.1 Tunneling Results

Field	Description
timestamp	Time of detection
entropy_value	Computed entropy
tunneling_flag	True/False
protocol	DNS or HTTP

4.3.2 Classification Results

Field	Description
timestamp	Classification time
attack_type	Brute force / port scan / malware download
behavior	Bot or human

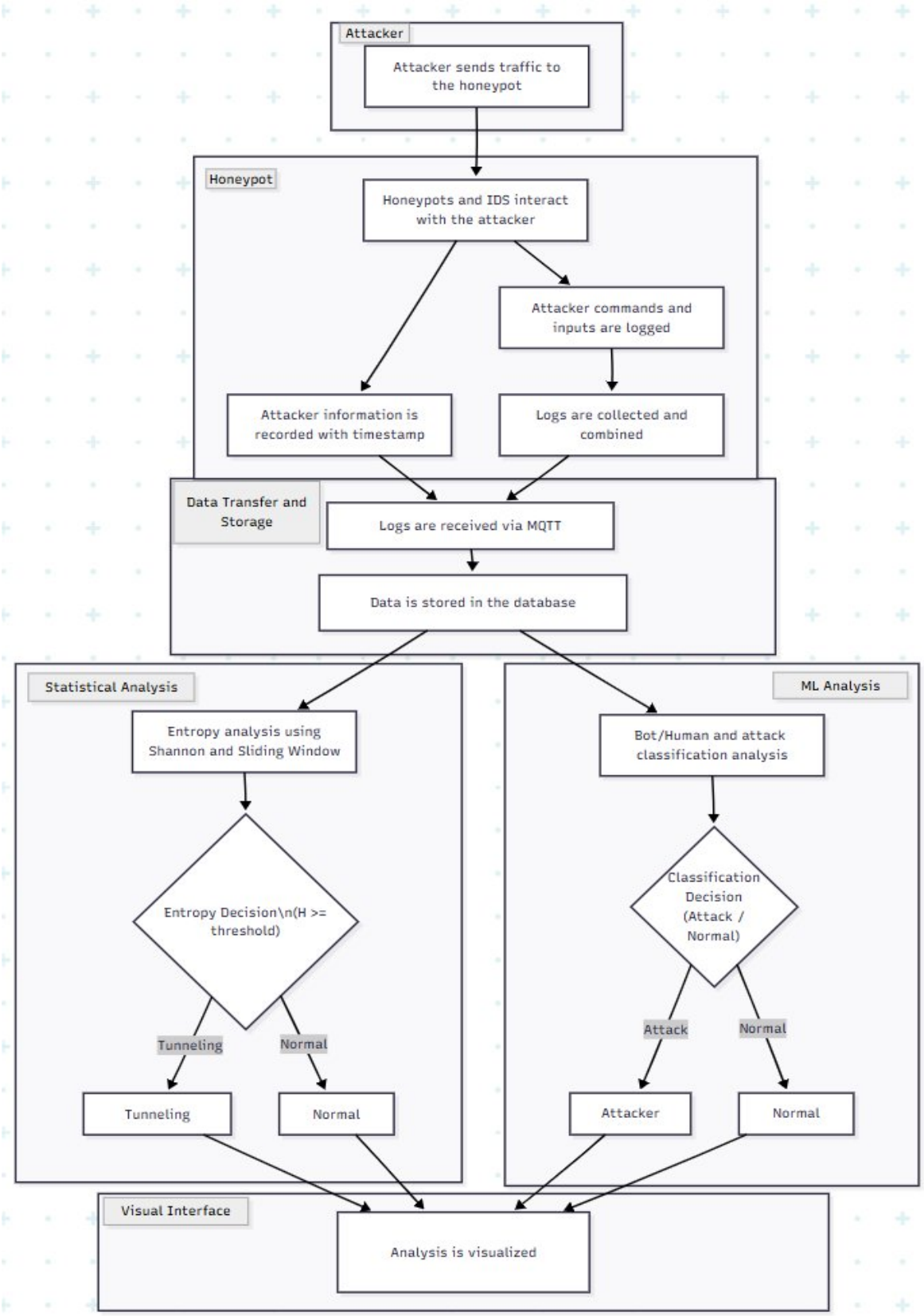
4.3.3 Performance Metrics

Metric	Description
accuracy	Accuracy of model
f1_score	F1-score
false_positive_rate	FPR
precision	Precision
recall	Recall

4.4 Data Flow Diagram (DFD)

This section will include a DFD showing:

- Attackers → Honeypots
- Honeypots → Log Aggregation
- Aggregation → MQTT → Database
- Database → Preprocessing → Feature Extraction
- Features → Statistical/ML Analysis
- Results → InfluxDB → Dashboard


Attacker
Attacker sends traffic to the honeypot
Honeypot
Honeypots and IDS interact with the attacker
Attacker commands and inputs are logged
Attacker information is recorded with timestamp
Logs are collected and combined
Data Transfer and Storage
Logs are received via MQTT
Data is stored in the database
Statistical Analysis
Entropy analysis using Shannon and Sliding Window
Entropy Decision\n(H >= threshold)
Tunneling
Normal
Tunneling
Normal
ML Analysis
Bot/Human and attack classification analysis
Classification Decision (Attack / Normal)
Attack
Normal
Attacker
Normal
Visual Interface
Analysis is visualized

4.5 Data Dictionary (High-Level)

Below is the definition of all key data items used by analytics.

Entropy-Related Fields

Field	Meaning
character_entropy	Shannon entropy of DNS/HTTP strings
entropy_pattern	Sliding-window entropy trend

Traffic Characteristics

Field	Meaning
session_duration	Total time of attacker's session
query_frequency	Number of queries per unit time
uri_length	Length of HTTP request path
ip_diversity	Count of unique IPs observed
connection_intensity	Request/payload rate

Classification Outputs

Field	Meaning
attack_type	Label predicted by classifier
behavior	bot vs human

Time-Series Output Fields

Field	Meaning
tunneling_flag	Indicates tunneling detection
timestamp	Event time
metric_value	Value of analysis metric

5. Component Design

This section describes the internal components of each layer in the system architecture.
Each component directly corresponds to the functional requirements defined in the SRS. The aim of this section is to provide a detailed, application-independent view of how each subsystem operates and interacts with the others.

5.1 Honeypot Layer Components

The Honeypot Layer runs on the Raspberry Pi device and is responsible for capturing real attacker traffic.

5.1.1 Cowrie Component (SSH Honeypot)

Responsibilities:

- Simulate SSH/Telnet services.
- Capture attacker commands, login attempts, and interactive sessions.
- Generate timestamped logs.

Inputs:

- External SSH traffic from attackers.

Outputs:

- Command logs, session logs, authentication logs.

Interactions:

- Sends log entries to the Log Aggregation Component.

5.1.2 Honeytrap Component (HTTP/FTP Emulation)

Responsibilities:

- Emulate HTTP and FTP services.
- Record HTTP requests, attempted file downloads, and scanning activity.

Inputs:

- External HTTP/FTP requests.

Outputs:

- HTTP/FTP request logs, request metadata, URIs, headers.

Interactions:

- Sends log entries to the Log Aggregation Component.

5.1.3 Suricata Component (Network IDS)

Responsibilities:

- Monitor all network packets entering the Raspberry Pi.
- Detect suspicious traffic patterns and known attack signatures.
- Produce alert logs.

Inputs:

- Raw network packets (DNS, HTTP, TCP/UDP).

Outputs:

- IDS alerts, DNS logs, HTTP metadata.

Interactions:

- Sends logs to the Log Aggregation Component.

5.2 Data Transfer and Storage Components

This layer ensures continuous log ingestion and structured storage for analysis.

5.2.1 Log Aggregation Component

Responsibilities:

- Collect logs generated by Cowrie, Honeytrap, and Suricata.
- Standardize log format and metadata.
- Prepare entries for MQTT transmission.

Inputs:

- Raw logs from Honeypot Layer.

Outputs:

- Normalized log entries.

Interactions:

- Feeds data into the MQTT Publisher Component.

5.2.2 MQTT Publisher Component (Edge Device)

Responsibilities:

- Publish log entries to MQTT topics.
- Maintain continuous delivery of logs.
- Retry transmission during temporary connectivity issues (as required by SRS).

Inputs:

- Standardized log entries.

Outputs:

- MQTT messages sent to the central server.

Interactions:

- Communicates with the MQTT Broker Component.

5.2.3 MQTT Broker Component

Responsibilities:

- Receive MQTT messages from the Raspberry Pi.
- Forward them to the SQLite Storage Component.
- Maintain topic subscriptions for log ingestion.

Inputs:

- MQTT messages.

Outputs:

- Log records ready for storage.

Interactions:

- Sends messages to SQLite Storage Component.

5.2.4 SQLite Storage Component

Responsibilities:

- Parse MQTT-delivered logs.
- Validate formatting and required fields.
- Store logs in structured tables.

- Store extracted features for analysis.

Inputs:

- MQTT log entries.

Outputs:

- Structured log and feature tables.

Interactions:

- Provides data to preprocessing and analysis components.

5.3 Statistical Analysis Components (Tunneling Detection)

This subsystem defines DNS/HTTP protocol tunneling using entropy-based methods.

5.3.1 Entropy Calculation Component

Responsibilities:

- Compute Shannon entropy for DNS and HTTP content.
- Convert strings and queries into entropy scores.

Inputs:

- Preprocessed logs.

Outputs:

- Numerical entropy values.

Interactions:

- Supplies entropy values to Sliding Window Component.

5.3.2 Sliding Window Component

Responsibilities:

- Apply sliding-window segmentation to entropy sequences.
- Detect temporal entropy patterns and spikes.

Inputs:

- Entropy values.

Outputs:

- Entropy trends over time windows.

Interactions:

- Passes trends to Threshold Decision Component.

5.3.3 Threshold Decision Component

Responsibilities:

- Compare entropy patterns with defined thresholds.
- Classify traffic as **Tunneling** or **Normal**.

Inputs:

- Sliding-window results.

Outputs:

- Tunneling detection decisions.

Interactions:

- Sends decisions to the Time-Series Storage Component (InfluxDB).

5.4 Machine Learning Analysis Components

This subsystem classifies attack behavior using supervised and anomaly-detection models.

5.4.1 Preprocessing Component

Responsibilities:

- Clean and normalize raw logs.
- Extract relevant fields for ML models.

Inputs:

- SQLite raw logs.

Outputs:

- Cleaned, structured datasets.

Interactions:

- Provides data to the Feature Extraction Component.

5.4.2 Feature Extraction Component

Responsibilities:

- Compute measurable traffic features:
    - session duration
    - query frequency
    - character entropy
    - URI length
    - IP diversity
    - connection intensity
- Store features in SQLite.

Inputs:

- Preprocessed logs.

Outputs:

- Numerical feature vectors.

Interactions:

- Supplies features to Classification Component.

5.4.3 Classification Component

Responsibilities:

- Train and apply ML models.
- Detect attack types:
    - brute force
    - port scan
    - malware download
    - bot vs human
- Produce evaluation metrics (accuracy, F1-score, false positive rate, recall, precision).

Inputs:

- Extracted features.

Outputs:

- Attack labels and performance metrics.

Interactions:

- Sends output to Time-Series Storage Component.

5.5 Visualization Components

This layer stores analysis results and provides dashboards for researchers.

5.5.1 InfluxDB Time-Series Storage Component

Responsibilities:

- Store tunneling decisions.
- Store ML classification results.
- Store performance metrics.
- Organize all data as time-series entries.

Inputs:

- Tunneling detection results.
- Classification results.
- Evaluation metrics.

Outputs:

- Queryable time-series datasets.

Interactions:

- Visualized through Grafana/Superset.

5.5.2 Dashboard Visualization Component (Grafana / Superset)

Responsibilities:

- Display tunneling events and attack classifications.
- Provide geographic analysis of attacker origins.
- Show bot vs human behavior distribution.
- Present accuracy, F1-score, false positive rate, etc.
- Enable time-based filtering and interactive exploration.

Inputs:

- Time-series data from InfluxDB.

Outputs:

- Visual dashboards for researchers and analysts.

6. Algorithm Design

This section explains the algorithms used in the system for tunneling detection and machine-learning-based attack classification.
All algorithms are directly based on the analytical descriptions in the SRS and do not introduce additional techniques or models.

The system uses two complementary analysis approaches:

1. Statistical (Entropy-Based) Tunneling Detection
2. Machine Learning Attack Classification

6.1 Entropy Calculation Algorithm

Shannon entropy is used to quantify randomness in DNS and HTTP traffic.
High entropy values may indicate encoded, encrypted, or artificially generated tunnel traffic.

Purpose:

To convert request strings (DNS queries, HTTP URIs) into a numerical entropy score.

Input:

- Preprocessed DNS and HTTP strings
- Example fields: domain names, subdomains, URI paths

Output:

- character_entropy: a numeric value representing randomness

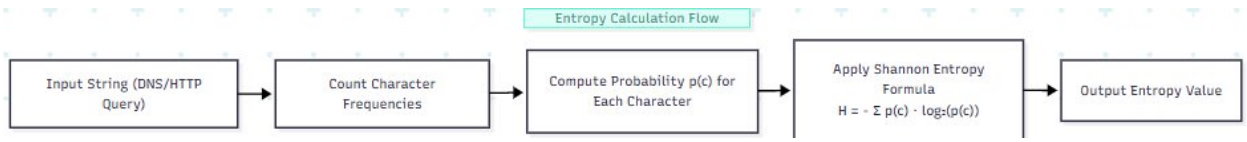
Algorithm Steps (High-Level):

1. Extract the string from the log entry
   (DNS query name, HTTP URI, request path)
2. Count character frequencies
   - For each unique character
   - Compute frequency: f(c) / N

3. Apply Shannon entropy formula:

$$H = -\sum p(c) \log_2 p(c)$$

4. Store entropy as part of the feature set
5. Provide entropy values to the Sliding Window module



6.2 Sliding Window Algorithm

This algorithm observes how entropy changes across time or sequence windows.
DNS and HTTP tunneling often show consistent high entropy over consecutive packets.

Purpose:

To detect trends or spikes in entropy that suggest tunneling behavior.

Input:

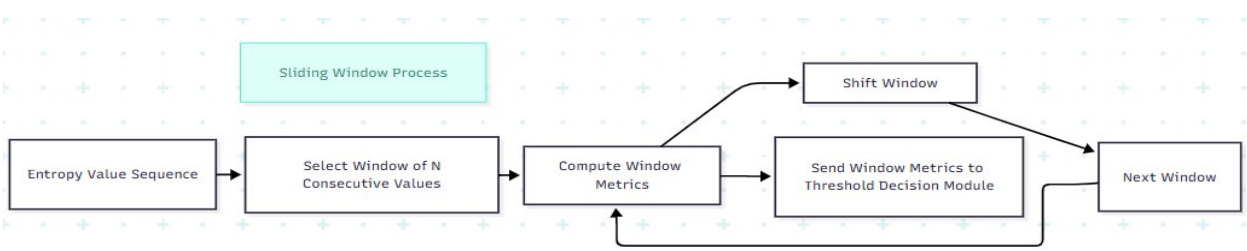
- Sequence of entropy values from Section 6.1

Output:

- entropy_pattern: time-based trend line

Algorithm Steps:

1. **Group entropy values** into windows (e.g., N consecutive queries)
2. **Compute window-level summaries:**
    - average entropy
    - max/min entropy
3. **Shift the window forward** by a fixed step
4. **Repeat for entire dataset**
5. **Provide results to the Threshold Decision Engine**



6.3 Tunneling Decision Algorithm

Using sliding-window entropy values, the system classifies traffic as normal or tunneling.

Purpose:

To identify DNS/HTTP tunneling using entropy thresholds.

Input:

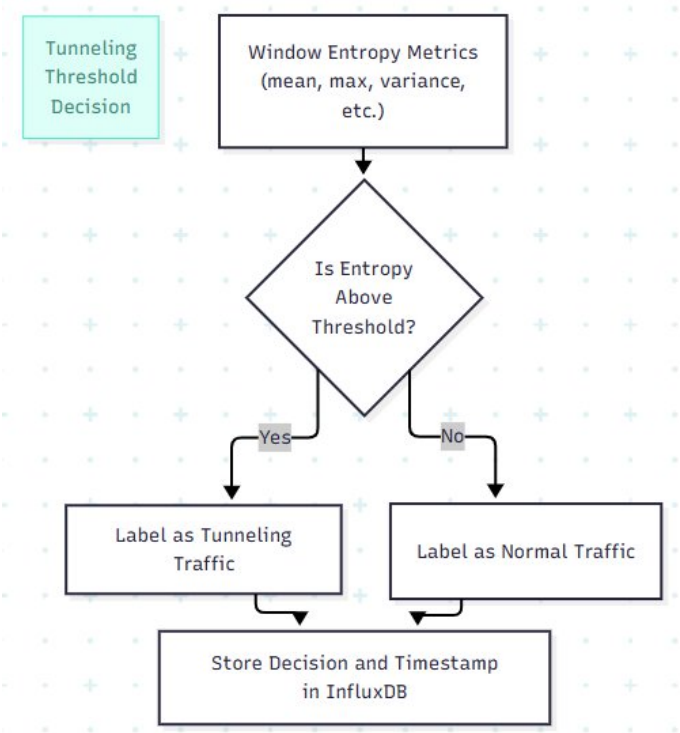
- Sliding-window entropy summaries
- Pre-determined entropy thresholds (defined during analysis)

Output:

- tunneling_flag = True/False

Algorithm Steps:

1. Receive window entropy metrics
2. Compare against threshold (e.g., H ≥ threshold)
3. If entropy consistently high:
    - Label window as Tunneling
    - Else: Label as Normal Traffic
4. Store detection result in InfluxDB



6.4 Machine Learning Classification Workflow

This algorithm identifies cyber attack types and behavioral profiles using numerical features.

Purpose:

To classify attacker behavior using supervised or anomaly-detection learning models.

Input:

- Feature vectors extracted from logs:
    - session duration
    - query frequency
    - character entropy
    - URI length
    - connection intensity
    - IP diversity

Output:

- attack_type (brute force / port scan / malware download / etc.)
- behavior (bot/human)
- performance metrics (accuracy, F1-score, false positive rate)

6.4.1 Feature Engineering Algorithm

Steps:

1. Clean and normalize numerical values
2. Extract independent variables from each log entry
3. Store feature vectors in SQLite
4. Provide features to ML classifier

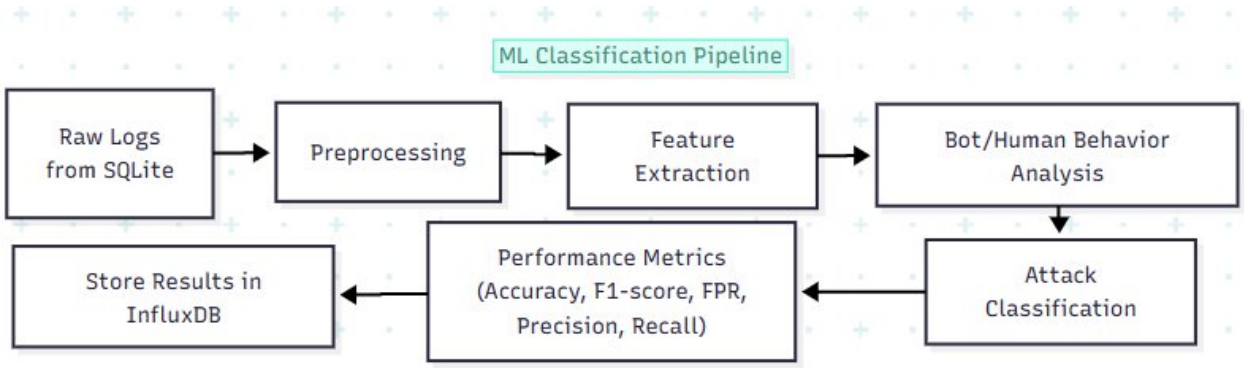
6.4.2 Classification Algorithm

Steps:

1. **Train the model**
   (supervised and/or anomaly-detection)
2. **Provide training data:**
    - extracted features

- labels (if available)

3. **Evaluate the model:**
    - accuracy
    - F1-score
    - false positive rate
    - precision / recall
4. **Predict attack type** for new events
5. **Predict bot vs human behavior**
6. **Generate output:**
    - attack_type
    - performance metrics
7. **Store predictions in InfluxDB**



6.5 Algorithm Dependencies

• Preprocessing → Feature extraction → (Tunneling Detection + ML Classification)

• Entropy calculation → Sliding window → Tunneling decision

• ML outputs (behavior, attack type, metrics) → InfluxDB → Visualization Layer

• Tunneling decisions → InfluxDB → Visualization Layer

Each algorithm consumes data from the previous step and feeds the next stage in the analysis pipeline.

7. User Interface Design

This section describes the user interface components of the system, focusing on visualization dashboards that present tunneling detection outputs, attack classification results, performance metrics, and time-series

patterns.
The interfaces are based strictly on the requirements defined in the SRS and rely on Grafana or Apache Superset for rendering.

7.1 Dashboard Overview

The system provides a web-based graphical interface through Grafana or Apache Superset.
The interface is designed for researchers, analysts, and system maintainers to:

- Monitor DNS/HTTP tunneling detections
- View classified attack events
- Visualize bot vs. human behavior
- Inspect geographic distribution of attacks
- Compare evaluation metrics (accuracy, F1-score, false positive rate)
- Explore time-series trends in attack behavior
- Filter results interactively over time windows

Dashboards are accessed through a web browser from the central analysis environment or a secure local network.

7.2 Dashboard Layout and Panels

The visualization layer consists of several panels arranged thematically.
The layout below describes the required panels, all derived from the SRS.

Panel Group 1 — Tunneling Detection

- Tunneling Events Over Time
- Entropy Trend Visualization (sliding-window)
- Protocol Type Breakdown (DNS vs HTTP)
- Normal vs Tunneling Classification Timeline

Panel Group 2 — Attack Classification

- Classified Attack Types
    - brute force
    - port scan
    - malware download
    - other behavior

- Bot vs Human Behavior Panel
- Attack Frequency Over Time

Panel Group 3 — Geographic Origin (if available)

- World Map / Country Heatmap
    - distribution of attacker IP origins
    - count per country

Panel Group 4 — Performance Metrics

- Accuracy Over Time
- F1-Score Over Time
- False Positive Rate
- Precision and Recall Panels

Panel Group 5 — Raw Data Exploration

- Table View of Tunneling Decisions
- Table View of Classified Attacks
- Feature-level exploration (entropy, URI length, etc.)

These panels are generated using InfluxDB queries.

7.3 Interaction Scenarios

The system supports several user interaction workflows:

Scenario 1 — Inspecting Tunneling Activity

1. The user opens the Tunneling Dashboard.
2. Selects a time range (e.g., last hour, last 24h, last 7 days).
3. Observes entropy patterns and tunneling decisions.
4. Cross-checks DNS/HTTP events to confirm tunneling behavior.

Scenario 2 — Analyzing Attack Types

1. User opens the Attack Classification Dashboard.
2. Views the distribution of attack types over time.
3. Filters results by timestamp or attack type (e.g., brute force only).
4. Examines bot vs human classification for detailed insights.

Scenario 3 — Evaluating Model Performance

1. The user navigates to the Performance Metrics Dashboard.
2. Compares the accuracy, F1-score, and false positive rate graphs.
3. Confirms whether the system targets (≥80% accuracy/F1-score) have been met.

Scenario 4 — Exploring Geographic Origins

1. User opens the Geo Dashboard.
2. Uses the map panel to view attacker origin distribution.
3. Identifies attack sources by region.

Scenario 5 — Raw Log/Feature Inspection

1. User accesses the raw data tables.
2. Sorts by timestamp, feature value, or event type.
3. Downloads or filters logs for deeper offline study.

7.4 UI Wireframe / Diagram

A high-level wireframe will be included to show:

- Placement of each panel
- Page structure (columns, rows)
- Interaction regions
- Navigation elements

8. Non-Functional Design

This section explains how the system design fulfills the non-functional requirements defined in the SRS. Each subsection corresponds directly to an NFR category and describes the design strategies used to ensure compliance.

8.1 Performance Design

The system is designed to meet the performance requirements for tunneling detection, machine-learning classification, data ingestion, and dashboard responsiveness.

Design Mechanisms:

- **Lightweight Data Transfer (MQTT):**
  MQTT ensures efficient, real-time transmission of logs from the Raspberry Pi, even under limited bandwidth.
- **Efficient Entropy and Sliding-Window Computation:**
  Algorithms operate on preprocessed data stored in SQLite, minimizing overhead.
- **Local Preprocessing Before Analysis:**
  Normalization and cleaning reduce computation during ML and statistical analysis.
- **Feature Storage in SQLite:**
  Speeds up ML model evaluation by avoiding repeated computation.
- **InfluxDB for Fast Time-Series Queries:**
  Enables interactive dashboard rendering with minimal delay.

SRS Links:

Supports NFRs: **PR-1, PR-2, PR-3, PR-4, PR-5**

8.2 Reliability Design

The system must reliably collect and analyze real attack traffic while ensuring continuous operation.

Design Mechanisms:

- **Continuous Honeypot Operation:**
  Cowrie, Honeytrap, and Suricata run persistently on the Raspberry Pi.
- **MQTT Retry Behavior:**
  The publisher resends logs in case of temporary network interruptions (as required by the SRS).
- **Structured SQLite Storage:**
  Data is validated before insertion to prevent corruption.
- **Decoupled Layers:**
  Failures in analysis or visualization layers do not affect log collection.
- **Support for Backup Datasets:**
  If insufficient real data is received, public datasets (CTU-13, CIC-IDS 2017) may be integrated as per SRS reliability requirements.

SRS Links:

Supports NFRs: **RL-1, RL-2, RL-3, RL-4**

8.3 Security Design

The project intentionally exposes the Raspberry Pi to the Internet; therefore, the architecture ensures isolation and safe operation.

Design Mechanisms:

- **DMZ + Guest Network Isolation:**
  Raspberry Pi is placed outside internal university networks.
- **No Attack Forwarding:**
  Honeypots operate strictly as decoys and do not act as pivot points.
- **Local Storage Protection:**
  SQLite and InfluxDB remain accessible only within the analysis environment.
- **Controlled Exposure:**
  The system follows the project's security requirement to safely collect malicious traffic without allowing it to affect other systems.

SRS Links:

Supports NFRs: **SEC-1, SEC-2, SEC-3, SEC-4, SEC-5**

8.4 Usability Design

The system interfaces are designed for researchers and analysts who require clear, interactive dashboards.

Design Mechanisms:

- **Grafana / Apache Superset Dashboards:**
  Visual panels clearly present tunneling events, attack types, and geographical origins.
- **Time-Based Filtering:**
  Users can select intervals for deeper analysis (hour/day/week).
- **Metric Comparison Panels:**
  Accuracy rate, false positive rate, and F1 score can be visualized.
- **Readable Layouts:**
  Dashboards follow logical grouping: tunneling, classification, geography, metrics.

SRS Links:

Supports NFRs: **US-1, US-2, US-3, US-4**

8.5 Maintainability Design

The system is configured to allow updates, improvements, and debugging to be performed with minimal disruption.

Design Mechanisms:

- **Modular Architecture:**
  Each layer (honeypot, MQTT, storage, analysis, visualization) can be updated independently.
- **Structured Python Scripts:**
  Preprocessing, feature extraction, and ML modules are organized for easy modification.
- **Error Logging:**
  The system logs failures (e.g., MQTT transmission errors, corrupted entries).
- **Open-Source Publication:**
  The code and dataset will be published on GitHub for documentation and reproducibility purposes.

SRS Links:

Supports NFRs: **MT-1, MT-2, MT-3, MT-4**

8.6 Extensibility Design

The architecture supports adding future honeypots, analysis methods, or dashboard features.

Design Mechanisms:

- **Layered Architecture:**
  New components (e.g., additional decoy systems or IDS tools) can be added to the Honeypot Layer.
- **Modular ML Design:**
  New classifiers or improved ML algorithms can replace existing ones.
- **Scalable Visualization:**
  Additional metrics or panels can be added to Grafana/Superset without redesign.

SRS Links:

Supports NFRs: **EX-1, EX-2, EX-3**

8.7 Portability Design

The design ensures that the system can be used in various hardware and software environments without any modifications.

Design Mechanisms:

- **Cross-Platform Python Environment:**
  Analysis pipeline runs on any Linux-based central server.
- **Reproducible Honeypot Setup:**
  Raspberry Pi environment can be recreated on similar IoT hardware.
- **Modular Architecture:**
  The separation of honeypot, data transfer, storage, analysis and visualization components supports deployment on different Linux-based environments as described in the SRS portability requirements.

SRS Links:

Supports NFRs: **PT-1, PT-2, PT-3**

9. Deployment & Runtime View

This section provides the physical deployment structure of the system, along with detailed runtime scenarios that describe how components interact during operation.
The deployment view is based strictly on the SRS and reflects the real-world setup of the Raspberry Pi honeypot, the central analysis machine, the MQTT broker, and the visualization environment.

9.1 Physical Deployment Diagram

The system consists of two physical environments:

1. Edge Environment (Raspberry Pi)

Running:

- Cowrie (SSH honeypot)
- Honeytrap (HTTP/FTP emulation)
- Suricata IDS
- Log aggregation service
- MQTT Publisher

Network placement:

- Exposed to the Internet via **DMZ / Guest Network**
- Isolated from internal networks (per SRS security requirements)

2. Central Analysis Environment

Running:

- MQTT Broker

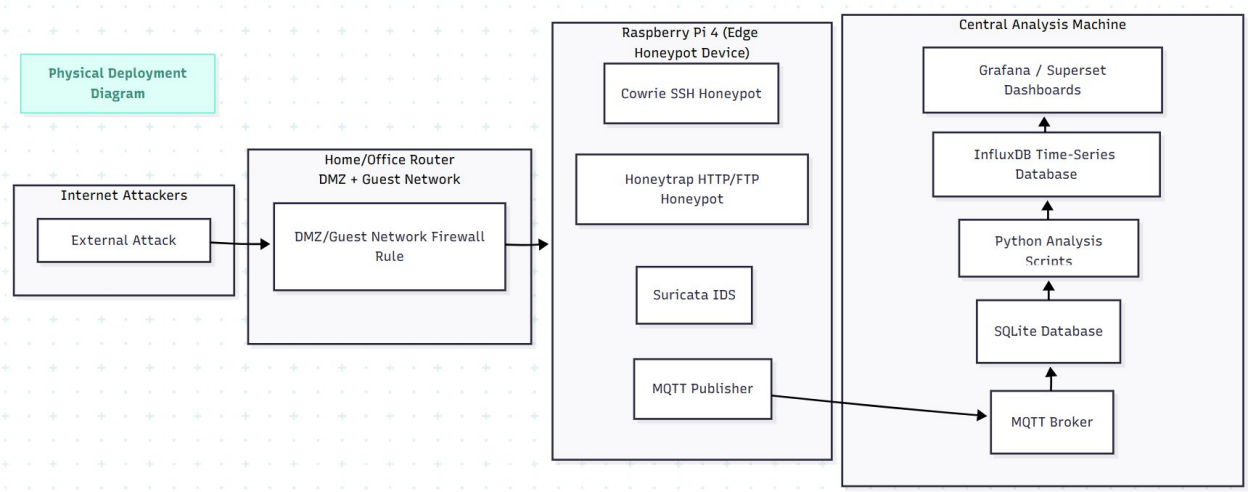
- SQLite database
- Python analysis scripts (preprocessing, feature extraction, ML, entropy detection)
- InfluxDB time-series storage
- Grafana / Superset dashboards

Network placement:

- Local secure network (not exposed to attackers)

3. Router / Network Layer

- Provides DMZ/Guest network for safe exposure
- Forwards attacker traffic only to the Raspberry Pi
- Ensures isolation from other hosts



9.2 Runtime Scenarios (Sequence Diagrams)

The following runtime scenarios describe the operational flow of the system during key processes.

9.2.1 Runtime Scenario 1 — Attack Event to Dashboard (Full Pipeline)

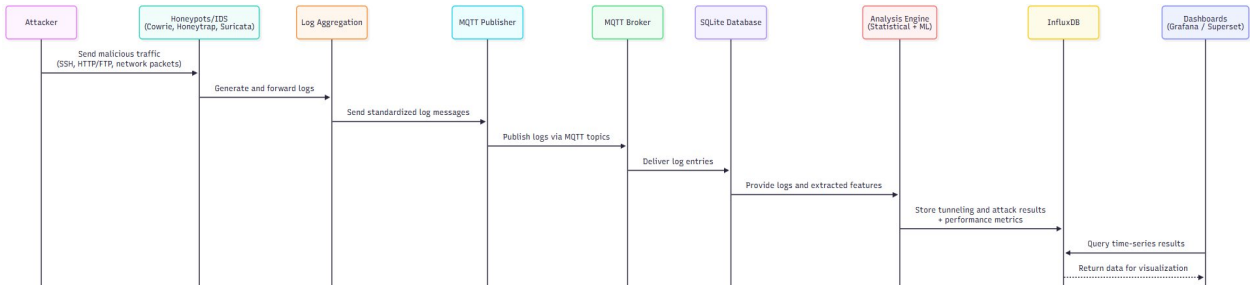
Purpose:
To describe how a single attack event travels through the entire system—from the attacker all the way to the visualization dashboard.

Sequence Steps:

1. Attacker sends SSH/HTTP/DNS traffic to the Raspberry Pi (DMZ).
2. Honeypot Layer receives the traffic.

3. Cowrie/Honeytrap/Suricata generate log entries.
4. Log Aggregator collects and standardizes logs.
5. Logs are forwarded to the MQTT Publisher.
6. Publisher sends logs to the MQTT Broker.
7. Broker delivers logs to the SQLite Storage module.
8. Logs are parsed and inserted into the database.
9. Preprocessing extracts required fields.
10. Feature Extraction computes session duration, entropy, URI length, etc.
11. Statistical & ML Analyses classify tunneling or attack type.
12. Analysis results are saved to InfluxDB.
13. Dashboards visualize updated results.

Sequence Diagram: Full Attack Event Flow



9.2.2 Runtime Scenario 2 — DNS/HTTP Tunneling Detection Flow

Purpose:
To describe the statistical analysis-only pipeline.

Sequence Steps:

1. SQLite provides preprocessed DNS/HTTP entries.
2. Entropy Calculation Module computes Shannon entropy for each request.
3. Entropy is passed to Sliding Window Module.
4. The sliding window computes time-based entropy trends.
5. Threshold Decision Engine compares trends to tunneling thresholds.
6. Labels traffic as Tunneling or Normal.

7. Results are written to InfluxDB.
8. Dashboard updates the tunneling visualization.

9.2.3 Runtime Scenario 3 — Machine Learning Attack Classification Flow

Purpose:
To describe how ML-based classification operates on incoming data.

Sequence Steps:

1. SQLite provides feature dataset (entropy, URI length, session duration, etc.).
2. Preprocessing Module cleans and normalizes values.
3. Feature Extraction Module computes final feature vectors.
4. Feature vectors are passed to the Classification Module.
5. Model produces:
    - attack_type (e.g., brute force, port scan)
    - bot/human behavior
    - performance metrics (accuracy, F1-score...)
6. Results are stored in InfluxDB.
7. Dashboard visualizes ML results and metrics.

10. SRS Traceability

This section maps the functional and non-functional requirements of the SRS to the corresponding design elements of the SDD.
The goal here is to ensure that the design fully meets the requirements and that all architectural, data, algorithmic, and interface decisions are justified by the SRS.

10.1 Functional Requirements Traceability

Table 10.1 – Mapping SRS Functional Requirements (FR) to SDD Sections

SRS Requirement ID	SRS Requirement Description	SDD Section(s) Addressing It
FR-1.1	Run Cowrie for SSH logging	5.1.1, 3.2.2
FR-1.2	Run Honeytrap for HTTP/FTP	5.1.2, 3.2.2
FR-1.3	Run Suricata for IDS and packet analysis	5.1.3, 3.2.2

SRS Requirement ID	SRS Requirement Description	SDD Section(s) Addressing It
FR-1.4	Timestamp and store all attacker interactions	3.2.2, 5.1.1–5.1.3, 4.2
FR-1.5	Operate Raspberry Pi in DMZ/Guest Network	3.2.1, 9.1
FR-2.1	Collect logs from all honeypots	5.2.1
FR-2.2	Merge/organize logs for MQTT transmission	5.2.1
FR-2.3	Include required metadata (timestamp, etc.)	5.2.1, 4.2
FR-3.1	Transmit logs to central server via MQTT	5.2.2, 5.2.3, 3.2.3
FR-3.2	Ensure uninterrupted forwarding	5.2.2
FR-3.3	Retry in case of connectivity issues	5.2.2
FR-4.1	Store logs in SQLite database	5.2.4, 4.2
FR-4.2	Create tables for different log types	4.2.1–4.2.3
FR-4.3	Validate entries before storage	5.2.4
FR-5.1	Preprocess raw logs	5.4.1
FR-5.2	Remove invalid/corrupted records	5.4.1
FR-5.3	Normalize numerical features	5.4.1
FR-6.1	Compute Shannon entropy	5.3.1, 6.1
FR-6.2	Extract defined features	5.4.2, 6.4.1
FR-6.3	Store extracted features in SQLite	5.4.2, 4.2.3
FR-7.1	Apply Shannon entropy to DNS/HTTP	5.3.1, 6.1
FR-7.2	Apply sliding-window analysis	5.3.2, 6.2
FR-7.3	Classify traffic as tunneling/normal	5.3.3, 6.3
FR-7.4	Aim for ≥80% tunneling detection accuracy	8.1
FR-8.1	Train ML models for attack classification	5.4.3, 6.4.2

SRS Requirement ID	SRS Requirement Description	SDD Section(s) Addressing It
FR-8.2	Classify brute force, port scan, malware, bot/human	5.4.3
FR-8.3	Evaluate classifier accuracy using F1-score	6.4.2, 8.1
FR-8.4	Aim for ≥80% F1-score	8.1
FR-9.1	Store analysis results in InfluxDB	5.5.1, 4.3
FR-9.2	Store timestamps and metrics with results	4.3
FR-9.3	Support InfluxDB queries for dashboards	5.5.1
FR-10.1	Provide visual dashboards	7.1, 7.2
FR-10.2	Display attack timestamps, origins, bot/human	7.2
FR-10.3	Compare metrics (accuracy, F1-score, FPR)	7.2
FR-10.4	Generate visual summaries	7.2, 7.4

10.2 Non-Functional Requirements Traceability

Table 10.2 – Mapping SRS NFRs to SDD Design Decisions

NFR ID	Requirement Description	SDD Section Addressing It
PR-1	≥80% tunneling detection accuracy	8.1
PR-2	≥80% F1-score for ML classification	8.1
PR-3	Efficient entropy/sliding-window analysis	6.1–6.3, 8.1
PR-4	SQLite must handle log volume	4.2, 8.1
PR-5	Dashboards must load quickly	5.5.2, 7.2, 8.1
RL-1	Continuous honeypot operation	3.2.2, 5.1
RL-2	Reliable MQTT delivery	5.2.2–5.2.3, 8.2

NFR ID	Requirement Description	SDD Section Addressing It
RL-3	No corruption of stored logs	5.2.4, 4.2
RL-4	Support for backup datasets	8.2
SEC-1	DMZ/Guest isolation	3.2.1, 9.1, 8.3
SEC-2	No link to production networks	9.1, 8.3
SEC-3	Restricted access to log data	8.3
SEC-4	Honeypots do not forward attacks	3.2.2, 8.3
SEC-5	Controlled exposure to the Internet	9.1, 8.3
US-1	Clear visualization of results	7.1, 7.2
US-2	Time-based filtering	7.2
US-3	Distinguish tunneling/normal & attack types	7.2
US-4	Display geographic/bot-human data	7.2
MT-1	Modular honeypot/MQTT/DB configuration	3.2, 5.1–5.2, 8.5
MT-2	Maintainable Python scripts	5.4, 8.5
MT-3	Error logging	5.2.2, 8.5
MT-4	Code + dataset published open-source	8.5
EX-1	Support additional honeypots/IDS tools	3.2.2, 8.6
EX-2	Support additional ML models	5.4.3, 8.6
EX-3	Extend dashboards with new metrics	7.2, 8.6
PT-1	Run on any Linux-based server	8.7
PT-2	Platform-independent codebase	8.7
PT-3	Reproducible honeypot setup	5.1, 8.7